

REGULAR LANGUAGES

a.y. 2025-2026

1 / 109

Recognizing languages

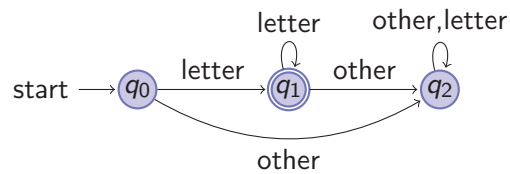
- $\mathcal{L} = \{a^n b^n \mid n > 0\}$ is a free language
- Stack seem ideal to recognize words in $\mathcal{L}$
- Read the string one symbol at a time: push a s onto stack; pop one symbol for each b and reject further a s; check emptiness
- Do we really need all that “counting” to generate, e.g., arbitrary strings over the English alphabet?

2 / 109

Recognizing languages

ARBITRARY STRINGS OVER THE ALPHABET

- Do we really need all that “counting” to generate, e.g., arbitrary strings over the alphabet?
- $S \rightarrow a \mid b \mid \dots \mid z \mid aS \mid bS \mid \dots \mid zS$
- A state machine is enough



3 / 109

Regular languages

- Regular grammars: free grammars with productions of the form
 - $A \rightarrow a$
 - $A \rightarrow aB$
 - $A \rightarrow \epsilon$
- Regular expressions
- Nondeterministic finite state automata
- Deterministic finite state automata
- At the basis of lexical analysis

4 / 109

Regular expressions

- Fix an alphabet $\mathcal{A}$, and a number of operators
- Define regular expressions inductively
 - Base:
 - Every $a \in \mathcal{A}$ is a regular expression
 - ϵ is a regular expression
 - Step: If r_1 and r_2 are regular expressions then
 - (Alternation) $r_1 \mid r_2$ is a regular expressions
 - (Concatenation) $r_1 \cdot r_2$ is a regular expressions (written $r_1 r_2$)
 - (Kleene star) r_1^* is a regular expressions
 - (Parentheses) (r_1) is a regular expression

5 / 109

Regular expressions

DENOTED LANGUAGE

- Given a regular expression r over $\mathcal{A}$, the **language denoted** by r , written $\mathcal{L}(r)$, is also inductively defined on the structure of r
 - Base:
 - $\mathcal{L}(a) = \{a\}$ for every $a \in \mathcal{A}$
 - $\mathcal{L}(\epsilon) = \{\epsilon\}$
 - Step:
 - If $r = r_1 \mid r_2$ then $\mathcal{L}(r) = \mathcal{L}(r_1) \cup \mathcal{L}(r_2)$
 - If $r = r_1 r_2$ then $\mathcal{L}(r) = \{w_1 w_2 \mid w_1 \in \mathcal{L}(r_1) \text{ and } w_2 \in \mathcal{L}(r_2)\}$
 - If $r = r_1^*$ then

$$\mathcal{L}(r) = \{\epsilon\} \cup \{w_1 w_2 \dots w_k \mid k \geq 1 \text{ and } \forall i : 1 \leq i \leq k. w_i \in \mathcal{L}(r_1)\}$$
 - If $r = (r_1)$ then $\mathcal{L}(r) = \mathcal{L}(r_1)$

6 / 109

Regular expressions

CONVENTIONS

- Kleene star has highest precedence, is left associative
- Concatenation has second highest precedence, is left associative
- Alternation has lowest precedence, is left associative
- Then $(a \mid bc^*)$ means
 - $(a \mid b(c^*))$
 - $(a \mid (b(c^*)))$

7 / 109

Regular expressions

EXAMPLES

- $\mathcal{L}(a \mid b) = \{a, b\}$
- $\mathcal{L}((a \mid b)(a \mid b)) = \{aa, ab, ba, bb\}$
- $\mathcal{L}(a^*) = \{a^n \mid n \geq 0\}$
- $\mathcal{L}(a \mid a^*b) = \{a\} \cup \{a^n b \mid n \geq 0\}$

8 / 109

Regular expressions

EXAMPLES

- $(a \mid b \mid \dots \mid z)(a \mid b \mid \dots \mid z)^*$ denotes the set of words over the alphabet
- $(0 \mid 1)^*0$ denotes all the binary numbers whose last digit is 0
- $b^*(abb^*)^*(a \mid \epsilon)$ denotes the set of words over $\{a, b\}$ with no consecutive occurrences of a
- $(a \mid b)^*aa(a \mid b)^*$ denotes the set of words over $\{a, b\}$ with at least one occurrence of two consecutive a 's

9 / 109

Finite states automata

- Finite state automata are used to decide whether a word belongs to the language denoted by a regular expression
- Nondeterministic Finite state Automata (NFA)
- Deterministic Finite state Automata (DFA)

10 / 109

Nondeterministic finite state automata

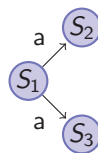
- An NFA is a tuple $(S, \mathcal{A}, \text{move}_n, s_0, F)$
- Where
 - S is a set of states
 - $\mathcal{A}$ is an alphabet with $\epsilon \notin \mathcal{A}$
 - $s_0 \in S$ is the **initial** state
 - $F \subseteq S$ is the set of **final** (or **accepting**) states
 - $\text{move}_n : S \times (\mathcal{A} \cup \{\epsilon\}) \rightarrow 2^S$ is the transition function

11 / 109

Nondeterministic finite state automata

GRAPHICAL REPRESENTATION

- $(S, \mathcal{A}, \text{move}_n, s_0, F)$ is represented as a directed graph
- Where
 - Nodes represent states
 - The initial state is identified by an incoming arrow
 - Final states are identified by a double circle
 - Edges represent the transition function
 - E.g., suppose $\text{move}_n(S_1, a) = \{S_2, S_3\}$, then

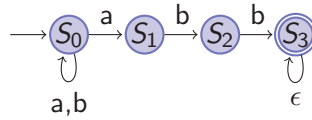


12 / 109

Nondeterministic finite state automata

EXAMPLE

An NFA



Tabular representation of its transition function

	ϵ	a	b
S_0	$\emptyset$	$\{S_0, S_1\}$	$\{S_0\}$
S_1	$\emptyset$	$\emptyset$	$\{S_2\}$
S_2	$\emptyset$	$\emptyset$	$\{S_3\}$
S_3	$\{S_3\}$	$\emptyset$	$\emptyset$

13 / 109

Nondeterministic finite state automata

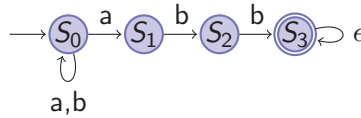
ACCEPTED LANGUAGES

- The NFA $\mathcal{N}$ **accepts** (or **recognizes**) w iff there exists **at least** a path spelling w from its initial state to one of its final states
- Recall
 - $\epsilon\epsilon$ spells ϵ
 - $a\epsilon$ spells a
 - ϵa spells a
- The language **accepted** (or **recognized**) by $\mathcal{N}$, written $\mathcal{L}(\mathcal{N})$, is the set of all the strings accepted by $\mathcal{N}$

14 / 109

Nondeterministic finite state automata

EXAMPLE



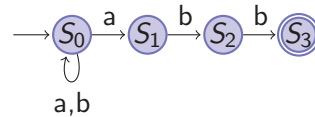
Accepted language

$$\mathcal{L}((a \mid b)^*abb)$$

15 / 109

Nondeterministic finite state automata

EXAMPLE



Accepted language

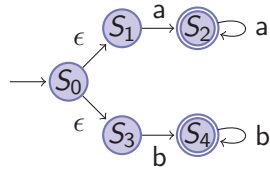
$$\mathcal{L}((a \mid b)^*abb)$$

Same language as the previous example

16 / 109

Nondeterministic finite state automata

EXAMPLE



Accepted language

$$\mathcal{L}(aa^* \mid bb^*)$$

Thompson's construction

An algorithm to construct an NFA $\mathcal{N}$ from a regular expression r in such a way that

$$\mathcal{L}(\mathcal{N}) = \mathcal{L}(r)$$

Thompson's construction

The construction is based on the inductive definition of regular expressions

- Base: r is either ϵ or a symbol of the alphabet
 - Define an NFA to recognize $\mathcal{L}(\epsilon)$
 - Define an NFA to recognize $\mathcal{L}(a)$
- Step: r is either $r_1 \mid r_2$, or $r_1 r_2$, or r_1^* , or (r_1)
 - Given NFAs $\mathcal{N}_1$ and $\mathcal{N}_2$ such that $\mathcal{L}(\mathcal{N}_i) = \mathcal{L}(r_i)$ for $i = 1, 2$
 - Define an NFA to recognize $\mathcal{L}(r_1 \mid r_2)$
 - Define an NFA to recognize $\mathcal{L}(r_1 r_2)$
 - Define an NFA to recognize $\mathcal{L}(r_1^*)$
 - Define an NFA to recognize $\mathcal{L}((r_1))$

19 / 109

Thompson's construction

MAIN FEATURES

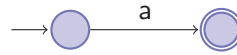
- Every step of the construction introduces 2 new states at most
 - The generated NFA has $2k$ states at most, where k is the number of symbols and of operators in the regular expression
- In every intermediate NFA there is
 - Exactly one final state
 - No edge incoming in the initial state
 - No edge outgoing from the final state

20 / 109

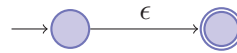
Thompson's construction

BASE

$$r = a$$



$$r = \epsilon$$

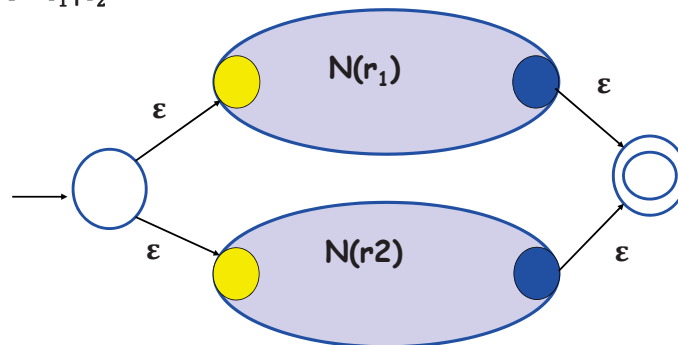


21 / 109

Thompson's construction

STEP: ALTERNATION

$$r = r_1 \mid r_2$$

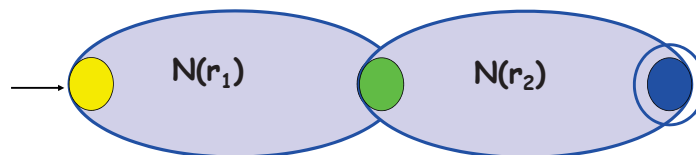


22 / 109

Thompson's construction

STEP: CONCATENATION

$$r = r_1 r_2$$



23 / 109

Thompson's construction

OBSERVATION

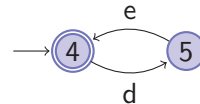
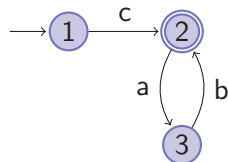
- Concatenation of regular expressions:
 - The construction is more efficient than linking the final node of $\mathcal{N}(r_1)$ with the initial node of $\mathcal{N}(r_2)$ with an ϵ -edge (see ϵ -closures below!)

24 / 109

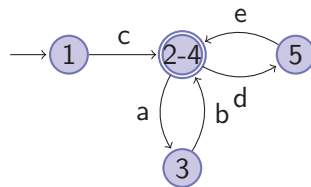
Thompson's construction

OBSERVATION

- Insisting on “No edge incoming in the initial state” and “No edge outgoing from the final state” guarantees that the constructed automaton accepts just $\mathcal{L}(r_1 r_2)$
- Think, e.g., of concatenating



- Into

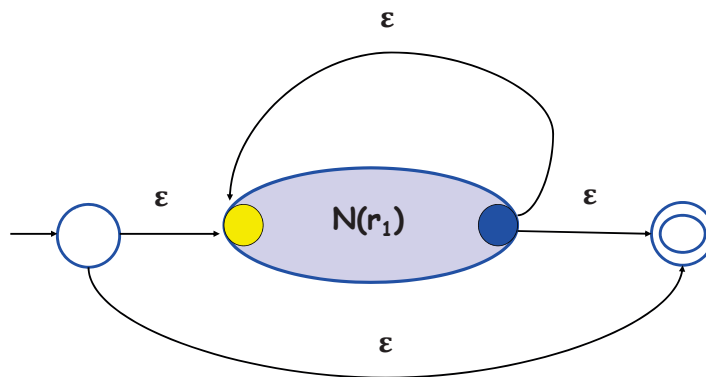


25 / 109

Thompson's construction

STEP: KLEENE STAR

$$r = r_1^*$$

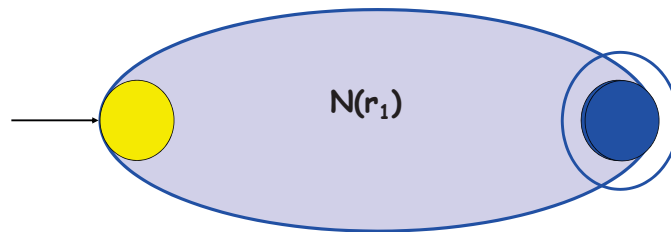


26 / 109

Thompson's construction

STEP: PARENTHESES

$$r = (r_1)$$

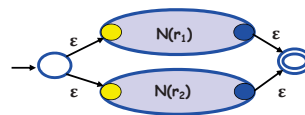


27 / 109

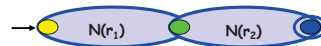
Thompson's construction

STEP: SUMMARY

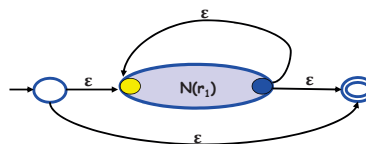
$$r = r_1 \mid r_2$$



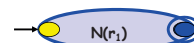
$$r = r_1 r_2$$



$$r = r_1^*$$



$$r = (r_1)$$



28 / 109

Thompson's construction

COMPLEXITY

- Constructs an NFA with n nodes and m edges
- Every step adds at most 2 states and 4 edges
- Every step has constant time
- There are $|r|$ steps
- Then:
 - **Space:** $n + m$ is $\mathcal{O}(|r|)$
 - **Time:** $\mathcal{O}(|r|)$

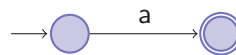
29 / 109

Thompson's construction

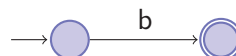
EXAMPLE OF APPLICATION

Take $r = (a \mid b)^*abb$

$a = r_1$



$b = r_2$



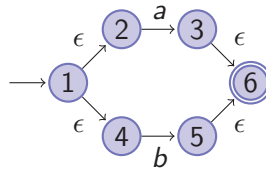
30 / 109

Thompson's construction

EXAMPLE OF APPLICATION

$$a \mid b = r_1 \mid r_2 = r_3$$

Apply Thompson's construction for alternation to the automata for r_1 and r_2 , and get the automaton for r_3



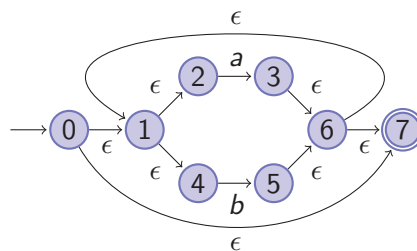
31 / 109

Thompson's construction

EXAMPLE OF APPLICATION

$$(a \mid b) = (r_3) = r_4 \qquad (a \mid b)^* = r_4^* = r_5$$

Apply Thompson's construction for parentheses and for Kleene star to the automaton for r_3 , and get the automaton for r_5



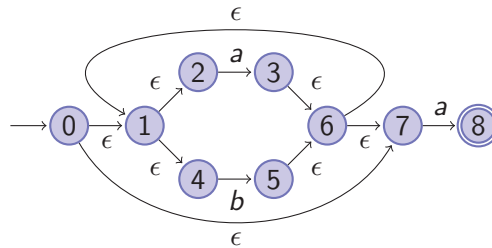
32 / 109

Thompson's construction

EXAMPLE OF APPLICATION

$$(a \mid b)^* a = r_5 r_1 = r_6$$

Apply Thompson's construction for concatenation to the automata for r_5 and for r_1 , and get the automaton for r_6



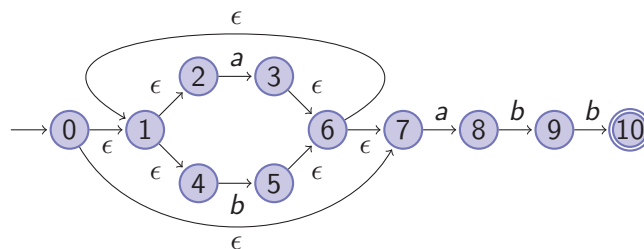
33 / 109

Thompson's construction

EXAMPLE OF APPLICATION

$$r = (a \mid b)^* abb$$

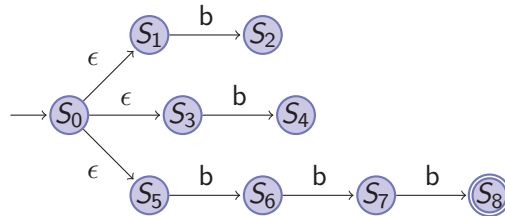
In two more steps



34 / 109

Simulation of NFAs

- Given a word w and an NFA $\mathcal{N}$ decide whether $w \in \mathcal{L}(\mathcal{N})$
- Let $w = bbb$



- Reading bbb from S_0 is the same as reading it from any of the states in $\{S_0, S_1, S_3, S_5\}$

35 / 109

Simulation of NFAs

ϵ -CLOSURE

- Let $(S, \mathcal{A}, \text{move}_n, s_0, F)$ be an NFA, t be a state in S , and T be a subset of states
- $\epsilon\text{-closure}(\{t\})$ is the set of states in S which are reachable from t by zero or more ϵ -transitions
- $\epsilon\text{-closure}(T) = \bigcup_{t \in T} \epsilon\text{-closure}(\{t\})$

36 / 109

Simulation of NFAs

COMPUTATION OF ϵ -CLOSURE

Data structures:

- Stack
- Boolean array “alreadyOn” of size $|S|$ to check in constant time whether a state t is onto the stack
- Bidimensional array to record move_n . Every entry (t, x) is a linked list containing all the states in $\text{move}_n(t, x)$

37 / 109

Simulation of NFAs

COMPUTATION OF ϵ -CLOSURE

```

foreach  $i=1, \dots, |S|$  do alreadyOn[i] = false ;
function closure( $t, \text{stack}$ )
    push  $t$  onto stack ;
    alreadyOn[t] = true ;
    foreach  $u \in \text{move}_n(t, \epsilon)$  do
        if not alreadyOn[u] then
            closure ( $u, \text{stack}$ ) ;

```

38 / 109

Simulation of NFAs

COMPLEXITY OF ϵ -CLOSURE

```

1 push t onto stack;
2 set alreadyOn[t] bit;
3 find next  $u \in \text{move}_n(t, \epsilon)$ ;
4 test alreadyOn[u] bit;

```

- Each of them takes constant time
- How many times are they repeated?

39 / 109

Simulation of NFAs

COMPLEXITY OF ϵ -CLOSURE

```

1 push t onto stack;
2 set alreadyOn[t] bit;
3 find next  $u \in \text{move}_n(t, \epsilon)$ ;
4 test alreadyOn[u] bit;

```

- Line1 & Line2: Executed at each invocation of closure (either initial or recursive)
 - Every state goes onto stack at most once (alreadyOn bit initially false, then set to true, never changed again)
- In aggregate, assuming that the NFA has n states and m edges, $\mathcal{O}(n)$

40 / 109

Simulation of NFAs

COMPLEXITY OF ϵ -CLOSURE

```

1 push t onto stack;
2 set alreadyOn[t] bit;
3 find next  $u \in \text{move}_n(t, \epsilon)$ ;
4 test alreadyOn[u] bit;

```

- Line3 & Line4: Executed at each invocation of closure for all $u \in \text{move}_n(t, \epsilon)$
 - In the worst case every state goes onto the stack, and every state has at least an ϵ -transition
- In aggregate, assuming that the NFA has n states and m edges, $\mathcal{O}(m)$

41 / 109

Simulation of NFAs

COMPLEXITY OF ϵ -CLOSURE

```

function closure( $t, \text{stack}$ )
|   push t onto stack ;
|   alreadyOn[t] = true ;
|   foreach  $u \in \text{move}_n(t, \epsilon)$  do
|       |   if not alreadyOn[u] then
|           |       closure (u, stack) ;
|

```

$$\mathcal{O}(n + m)$$

42 / 109

Simulation of NFAs

ALGORITHM

```

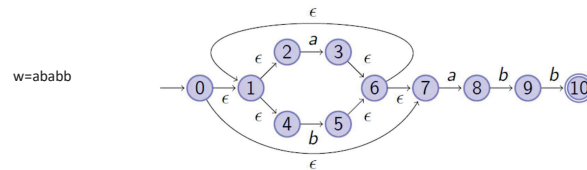
input           : NFA  $\mathcal{N} = (S, \mathcal{A}, \text{move}_n, s_0, F)$ , w$
output          : "yes" if  $w \in \mathcal{L}(\mathcal{N})$ , "no" otherwise
states =  $\epsilon$ -closure( $\{s_0\}$ ) ;
symbol = nextchar() ;
while symbol  $\neq$  $ do
    states =  $\epsilon$ -closure( $\bigcup_{t \in \text{states}} \text{move}_n(t, \text{symbol})$ ) ;
    symbol = nextchar() ;
if states  $\cap F \neq \emptyset$  then
    | return "yes" ;
else
    | return "no" ;

```

43 / 109

Simulation of NFAs

EXAMPLE



states	symbol	$\bigcup_{t \in \text{move}(t, \text{symbol})}$	ϵ -closure
T0 = {0,1,2,4,7}	a	{3,8}	{1,2,3,4,6,7,8}
T1 = {1,2,3,4,6,7,8}	b	{5,9}	{1,2,4,5,6,7,9}
T2 = {1,2,4,5,6,7,9}	a	{3,8}	T1
T1	b		T2
T2	b	{5,10}	{1,2,4,5,6,7,10}
T3 = {1,2,4,5,6,7,10}	\$		

44 / 109

Simulation of NFAs

COMPLEXITY

Dominant

```

while symbol  $\neq$  $ do
L1  |   states =  $\epsilon$ -closure( $\bigcup_{t \in \text{states}}$  moven(t, symbol)) ;
    |   symbol = nextchar() ;

```

Observe: variable *states* occurs both as left-hand side and in the right-hand side of the assignment at line L1

Simulation of NFAs

COMPLEXITY

Data structures:

- Two stacks for states:
 - currentStack for the current states ("states" on the right-side of the assignment at Line L1)
 - nextStack for the next states ("states" on the left-side of the assignment at Line L1)
- Boolean array "alreadyOn" of size $|S|$ to check in constant time whether a state is onto the stack
- Bidimensional array to record move_{*n*}. Every entry (*t*, *x*) is a linked list containing all the states in move_{*n*}(*t*, *x*)

Simulation of NFAs

COMPLEXITY, LINE L1

```

foreach t on currentStack do
|   :
|   ;                               /* populate nextStack */
foreach s on nextStack do
|   :
|   ;                               /* swap nextStack with currentStack */

```

47 / 109

Simulation of NFAs

COMPLEXITY, LINE L1

```

foreach t on currentStack do
|   foreach  $u \in \text{move}_n(t, \text{symbol})$  do
|   |   if not alreadyOn[u] then
|   |   |   closure(u, nextStack);
|   |   pop t from currentStack;
foreach s on nextStack do
|   pop s from nextStack ;
|   push s on currentStack ;
|   alreadyOn[s] = false ;

```

48 / 109

Simulation of NFAs

COMPLEXITY

- Assume that the NFA has n states and m edges
- For each while-cycle
 - Populating nextStack is $\mathcal{O}(n + m)$
 - Swapping the stacks is $\mathcal{O}(n)$
- One while-cycle costs $\mathcal{O}(n + m)$
- The simulation of w costs $\mathcal{O}(|w|(n + m))$
- If the NFA is obtained by Thompson's construction, $(n + m)$ is $\mathcal{O}(|r|)$, then simulating w is $\mathcal{O}(|w||r|)$

49 / 109

Wrap-up: Accepting regular languages by NFAs

- Take a regular expression r
- Apply Thomson's construction, get NFA $\leftarrow \mathcal{O}(|r|)$
- Simulate NFA $\leftarrow \mathcal{O}(|w||r|)$
- Overall: $\mathcal{O}(|w||r|)$

50 / 109

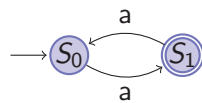
Deterministic finite state automata

- NFA: $(S, \mathcal{A}, \text{move}_n, s_0, F)$
 where $\text{move}_n : S \times (\mathcal{A} \cup \{\epsilon\}) \rightarrow 2^S$
- DFA: $(S, \mathcal{A}, \text{move}_d, s_0, F)$
 where $\text{move}_d : S \times \mathcal{A} \rightarrow S$
- In any DFA
 - There is no ϵ -transition
 - If move_d is **total**, then from every state there is **exactly one** a -transition for every $a \in \mathcal{A}$
 - If move_d is **partial**, then from every state there is **at most one** a -transition for every $a \in \mathcal{A}$

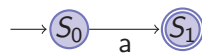
51 / 109

Deterministic finite state automata

- Take the alphabet $\{a\}$
- An example of DFA with total transition function is



- An example of DFA with partial transition function is



52 / 109

Simulation of DFAs

ACCEPTED LANGUAGES

The language recognized by a DFA $\mathcal{D}$, denoted by $\mathcal{L}(\mathcal{D})$, is the set of words w such that

- Either there is a path spelling $w = a_1 \cdots a_k$ with $k \geq 1$ from the initial state of $\mathcal{D}$ to some of its final states
- Or the initial state is also final and $w = \epsilon$

53 / 109

Simulation of DFAs

TOTAL TRANSITION FUNCTION

- Answer the question " $w \in \mathcal{L}(\mathcal{D})$?"
- Starting from the initial state, follow the path spelling w
- If the reached state is final, then return "yes"
- Otherwise return "no"

54 / 109

Simulation of DFAs

PARTIAL TRANSITION FUNCTION

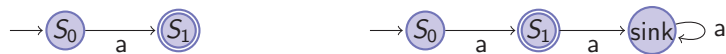
- Answer the question “ $w \in \mathcal{L}(\mathcal{D})$?”
- Starting from the initial state, begin following the path spelling $w = a_1 \cdots a_k$
- If for some a_i there is no target state, then return “no”
- If w is over and the reached state is final, then return “yes”
- Otherwise return “no”

55 / 109

Deterministic finite state automata

PARTIAL VS TOTAL TRANSITION FUNCTION

- Let $\mathcal{D}$ be a DFA with partial transition function
- Can you define a DFA $\mathcal{D}'$ with total transition function such that $\mathcal{L}(\mathcal{D}') = \mathcal{L}(\mathcal{D})$?
- Use a **sink**
 - Add a sink to the states of $\mathcal{D}$
 - Let the sink be the target of all the undefined transitions
 - For every symbol in the alphabet, add the sink with a self-loop



56 / 109

Subset construction

Given the NFA $\mathcal{N}$ construct a DFA $\mathcal{D}$ such that $\mathcal{L}(\mathcal{D}) = \mathcal{L}(\mathcal{N})$

Idea: Use ϵ -closure to map subsets of states of the NFA into one single state of the DFA

57 / 109

Subset Construction

- This is the first instance of a construction we will see again later in the course
- Define the initial state of the DFA
- Populate the collection of all its states while defining the transition function

```
foreach state  $T$  already collected do
  foreach symbol  $a$  in the alphabet do
    compute the plausible target of the  $a$ -transition from  $T$  ;
    call it  $T'$  ;
    if  $T'$  is already in the collection then
      | nothing else to do ;
    else
      | add  $T'$  to the collection ;
```

58 / 109

Subset Construction

```

input           : NFA  $\mathcal{N} = (S, \mathcal{A}, \text{move}_n, S_0, F)$ 
output          : DFA  $\mathcal{D} = (R, \mathcal{A}, \text{move}_d, T_0, E)$  such that  $\mathcal{L}(\mathcal{D}) = \mathcal{L}(\mathcal{N})$ 
 $T_0 = \epsilon\text{-closure}(\{S_0\});$ 
 $R = \{T_0\};$ 
set  $T_0$  as unmarked ;
while some  $T \in R$  is unmarked do
  mark  $T$  ;
  foreach  $a \in \mathcal{A}$  do
     $T' = \epsilon\text{-closure}(\bigcup_{t \in T} \text{move}_n(t, a)) ;$ 
    if  $T' \neq \emptyset$  then
       $\text{move}_d(T, a) = T' ;$ 
      if  $T' \notin R$  then
        add  $T'$  to  $R$  ;
        set  $T'$  as unmarked ;
  foreach  $T \in R$  do
    if  $(T \cap F) \neq \emptyset$  then set  $T \in E$  ;

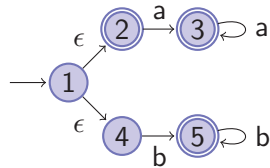
```

59 / 109

Subset construction

APPLICATION: EXAMPLE-1

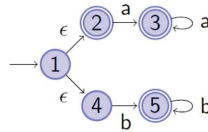
Take the NFA



60 / 109

Subset construction

APPLICATION: EXAMPLE-1



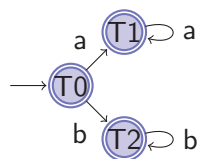
	a	b
$T0 = \epsilon\text{-closure}(\{1\}) = \{1,2,4\}$ % initial, final	$\epsilon\text{-closure}(\{3\}) = T1$	$\epsilon\text{-closure}(\{5\}) = T2$
$T1 = \{3\}$ % final	$\epsilon\text{-closure}(\{3\}) = T1$	---
$T2 = \{5\}$ % final	---	$\epsilon\text{-closure}(\{5\}) = T2$

61 / 109

Subset construction

APPLICATION: EXAMPLE-1

Get the DFA



62 / 109

Subset Construction

COMPLEXITY

```

input           : NFA  $\mathcal{N} = (S, \mathcal{A}, \text{move}_n, S_0, F)$ 
output          : DFA  $\mathcal{D} = (R, \mathcal{A}, \text{move}_d, T_0, E)$  such that  $\mathcal{L}(\mathcal{D}) = \mathcal{L}(\mathcal{N})$ 
 $T_0 = \epsilon\text{-closure}(\{S_0\});$ 
 $R = \{T_0\};$ 
set  $T_0$  as unmarked ;
while some  $T \in R$  is unmarked do
    mark  $T$  ;
    foreach  $a \in \mathcal{A}$  do
         $T' = \epsilon\text{-closure}(\bigcup_{t \in T} \text{move}_n(t, a)) ;$ 
        if  $T' \neq \emptyset$  then
             $\text{move}_d(T, a) = T' ;$ 
            if  $T' \notin R$  then
                add  $T'$  to  $R$  ;
                set  $T'$  as unmarked ;

foreach  $T \in R$  do
    if  $(T \cap F) \neq \emptyset$  then set  $T \in E$  ;
  
```

63 / 109

Subset Construction

COMPLEXITY

Dominant

```

while some  $T \in R$  is unmarked do
    ...;
    foreach  $a \in \mathcal{A}$  do
         $T' = \epsilon\text{-closure}(\bigcup_{t \in T} \text{move}_n(t, a)) ;$ 
        :
  
```

64 / 109

Subset Construction

COMPLEXITY

```

while some  $T \in R$  is unmarked do
  ...;
  foreach  $a \in \mathcal{A}$  do
     $T' = \epsilon\text{-closure}(\bigcup_{t \in T} \text{move}_n(t, a))$ ;
    ...;

```

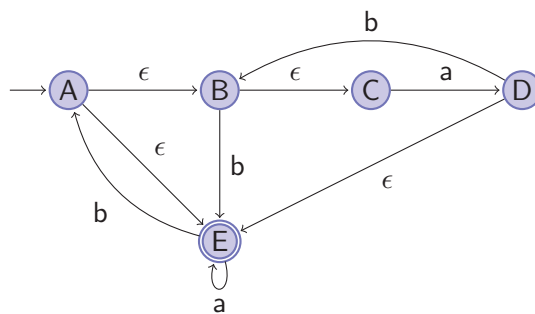
- Suppose the NFA has n states and m edges, and the DFA has n_d states
- **while** is repeated n_d times
- **foreach** is repeated $|\mathcal{A}|$ times
- $\epsilon\text{-closure}(\bigcup_{t \in T} \text{move}_n(t, a))$ costs $\mathcal{O}(n + m)$
- Then the subset construction is $\mathcal{O}(n_d \cdot |\mathcal{A}| \cdot (n + m))$
- The question is: how big can n_d be?

65 / 109

Subset construction

APPLICATION: EXAMPLE-2

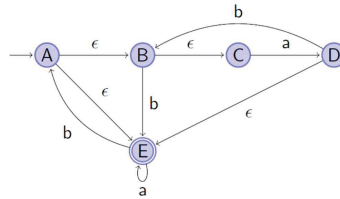
Take the NFA



66 / 109

Subset construction

APPLICATION: EXAMPLE-2



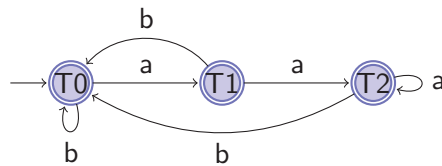
	a	b
$T0 = \epsilon\text{-closure}(\{A\}) = \{A, B, C, E\}$ % initial, final	$\epsilon\text{-closure}(\{D, E\}) = T1$	$\epsilon\text{-closure}(\{A, E\}) = \{A, B, C, E\} = T0$
$T1 = \{D, E\}$ % final	$\epsilon\text{-closure}(\{E\}) = T2$	$\epsilon\text{-closure}(\{A, B\}) = \{A, B, C, E\} = T0$
$T2 = \{E\}$ % final	$\epsilon\text{-closure}(\{E\}) = T2$	$\epsilon\text{-closure}(\{A\}) = T0$

67 / 109

Subset construction

APPLICATION: EXAMPLE-2

Get the DFA

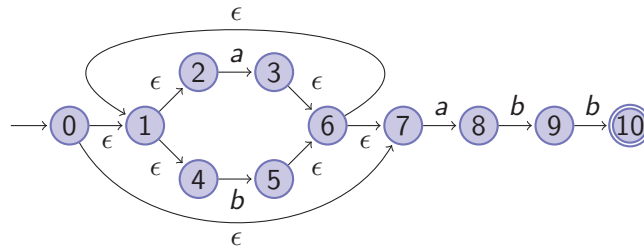


68 / 109

Subset construction

APPLICATION: EXAMPLE-3

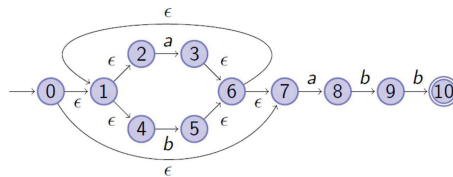
Take the NFA



69 / 109

Subset construction

APPLICATION: EXAMPLE-3



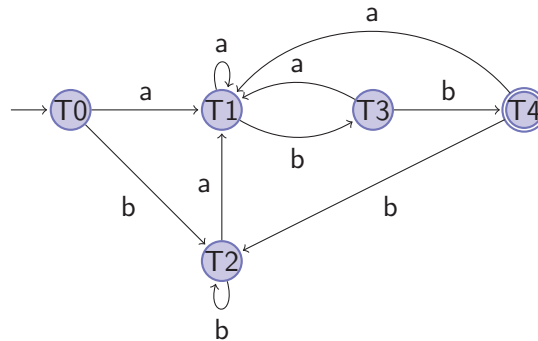
	a	b
T0 = ϵ -closure($\{0\}$) = {0,1,2,4,7} % initial	ϵ -closure($\{3,8\}$) = T1	ϵ -closure($\{5\}$) = T2
T1 = {1,2,3,4,6,7,8}	ϵ -closure($\{3,8\}$) = T1	ϵ -closure($\{5,9\}$) = T3
T2 = {1,2,4,5,6,7}	ϵ -closure($\{3,8\}$) = T1	ϵ -closure($\{5\}$) = T2
T3 = {1,2,4,5,6,7,9}	ϵ -closure($\{3,8\}$) = T1	ϵ -closure($\{5,10\}$) = T4
T4 = {1,2,4,5,6,7,10} % final	ϵ -closure($\{3,8\}$) = T1	ϵ -closure($\{5\}$) = T2

70 / 109

Subset construction

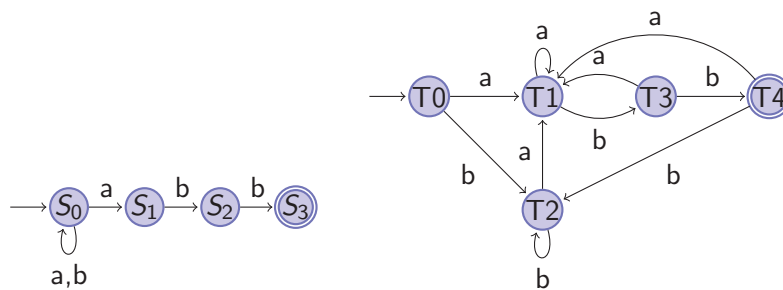
APPLICATION: EXAMPLE-3

Get the DFA



71 / 109

Subset construction



Both automata recognize $\mathcal{L}((a \mid b)^*abb)$

Can we get a smaller deterministic structure accepting the same language?

72 / 109

DFA minimization

Given the DFA $\mathcal{D}$ construct a minimal DFA $\mathcal{D}'$ such that $\mathcal{L}(\mathcal{D}') = \mathcal{L}(\mathcal{D})$

73 / 109

DFA minimization

INTUITION

Let $\mathcal{D} = (S, \mathcal{A}, \text{move}_\mathcal{D}, s_0, F)$ be a DFA and let $s_j \in S$. Then the *language recognized by s_j* , denoted $\mathcal{L}_\mathcal{D}(s_j)$ is the language accepted by $\mathcal{D}$ with s_j as initial state, namely the language recognized by $(S, \mathcal{A}, \text{move}_\mathcal{D}, s_j, F)$

To minimize a DFA, first partition the states of the DFA into equivalent classes

The equivalence of states consists in recognizing the same language:
Two states s and t are equivalent iff they recognize the same language

74 / 109

DFA minimization

PARTITION REFINEMENT: INIT

Let $\mathcal{D} = (S, \mathcal{A}, \text{move}_{\mathcal{D}}, s_0, F)$ be a DFA with **total transition function**

The partition is obtained by refinement of a coarse initial division of the states in two blocks

The initial blocks are

- $B_1 = F$
- $B_2 = S \setminus F$

Why this choice for the initial blocks?

$s \in B_1$ and $t \in B_2$ are not equivalent because $\epsilon \in \mathcal{L}_{\mathcal{D}}(s)$ and $\epsilon \notin \mathcal{L}_{\mathcal{D}}(t)$

DFA minimization

PARTITION REFINEMENT: STEP

- Check whether the blocks contain equivalent states
- If not so, refine the blocks by splitting them further
- Repeat the checking-refining steps up to the point that the partition cannot be further refined

DFA minimization

PARTITION REFINEMENT: SPLITTING BLOCKS

- If all the states in a block B are equivalent, then for every $a \in \mathcal{A}$ the target states of the a -transition from each state in B should belong to the same block B_j (with B_j not necessarily different from B)
- If, for some a and some B_j , a block B contains two states s and t such that

$$\text{move}_d(s, a) \in B_j \text{ and } \text{move}_d(t, a) \notin B_j$$
 then B can be **split by** (a, B_j)
- Splitting the block B by (a, B_j) amounts to replacing B by the two blocks
 - $\{s \in B \mid \text{move}_d(s, a) \in B_j\}$
 - $\{s \in B \mid \text{move}_d(s, a) \notin B_j\}$

77 / 109

DFA minimization

PARTITION REFINEMENT

```

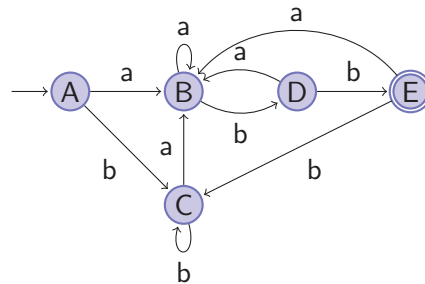
 $B_1 = F$  ;
 $B_2 = S \setminus F$  ;
 $P = \{B_1, B_2\}$  ;
while some  $B \in P$  can be split by some  $(a, B_j)$  do
  | Update  $P$  by removing  $B$  and adding the two blocks
  |  $\{s \in B \mid \text{move}_d(s, a) \in B_j\}$  and
  |  $\{s \in B \mid \text{move}_d(s, a) \notin B_j\}$ 

```

78 / 109

DFA minimization

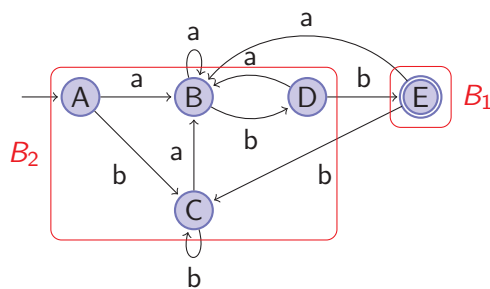
PARTITION REFINEMENT: EXAMPLE



79 / 109

DFA minimization

PARTITION REFINEMENT: EXAMPLE



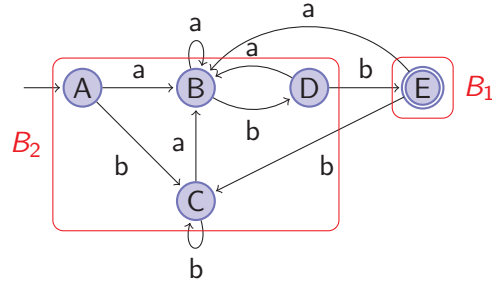
Initial Partition:

- $B_1 = \{E\}$
- $B_2 = \{A, B, C, D\}$

80 / 109

DFA minimization

PARTITION REFINEMENT: EXAMPLE



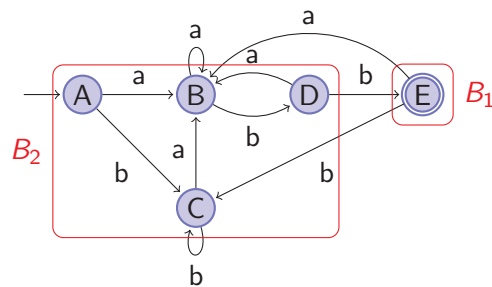
Block B can be split by (a, B_j) if for some $s, t \in B$

- $\text{move}_d(s, a) \in B_j$ and $\text{move}_d(t, a) \notin B_j$

81 / 109

DFA minimization

PARTITION REFINEMENT: EXAMPLE



We observe that

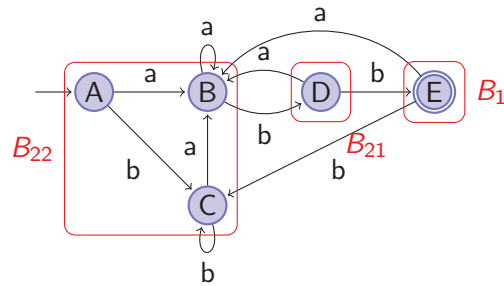
- $\text{move}_d(D, b) = E \in B_1$
- $\text{move}_d(A, b) = C \notin B_1$

And we split B_2 by (b, B_1)

82 / 109

DFA minimization

PARTITION REFINEMENT: EXAMPLE



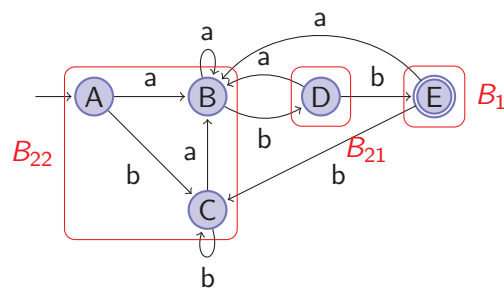
The current partition is

- $B_1 = \{E\}$
- $B_{21} = \{D\}$
- $B_{22} = \{A, B, C\}$

83 / 109

DFA minimization

PARTITION REFINEMENT: EXAMPLE



We observe that

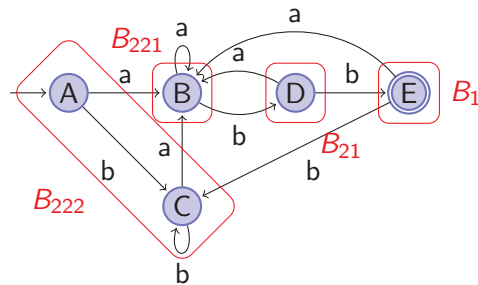
- $\text{move}_d(B, b) = D \in B_{21}$
- $\text{move}_d(A, b) = C \notin B_{21}$

And we split B_{22} by (b, B_{21})

84 / 109

DFA minimization

PARTITION REFINEMENT: EXAMPLE



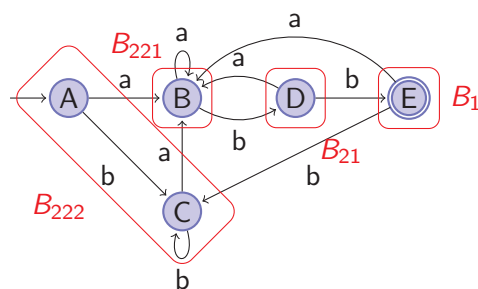
The current partition is

- $B_1 = \{E\}$
- $B_{21} = \{D\}$
- $B_{221} = \{B\}$
- $B_{222} = \{A, C\}$

85 / 109

DFA minimization

PARTITION REFINEMENT: EXAMPLE



No further split is possible

86 / 109

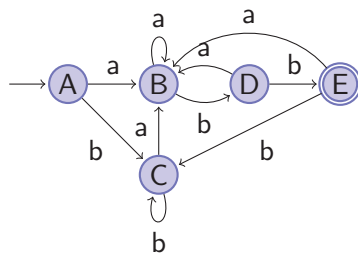
DFA Minimization

- Run partition refinement over a DFA with **total transition function**
- Each block is *temporarily* considered a state of the min-DFA
- The initial state of the min-DFA is the one represented by the block containing s_0
- The final states of the min-DFA are those represented by blocks containing final states of the original DFA
- The transition function of the min-DFA is derived from the original move_d as follows:
If $s \in B_i$ and $\text{move}_d(s, a) \in B_j$, then there is an a -transition in the min-DFA from the state representing block B_i to the state representing block B_j
- If any of the temporary state is dead, remove it, and remove all the transitions to/from it
- **Observe:** The min-DFA can have **partial** transition function

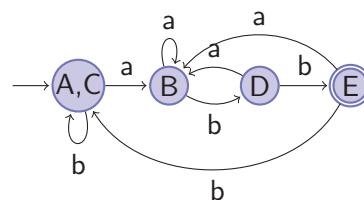
87 / 109

DFA minimization

MIN DFA AFTER THE PREVIOUS EXAMPLE OF PARTITION REFINEMENT



Partition: $\{A, C\}; \{B\}; \{D\}; \{E\}$

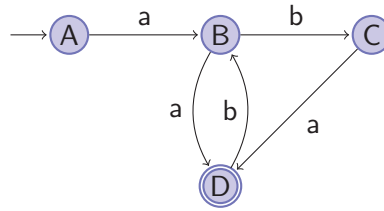


88 / 109

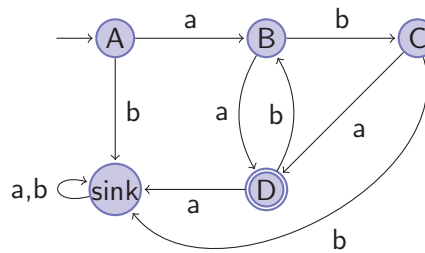
Training

EXAMPLE 1

- Minimize the DFA



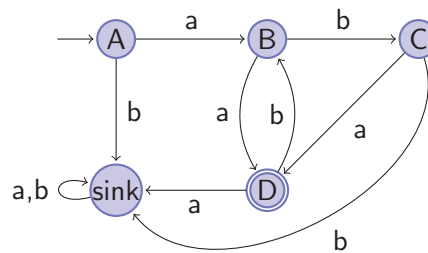
- The DFA has partial transition function, then take



89 / 109

Training

EXAMPLE 1

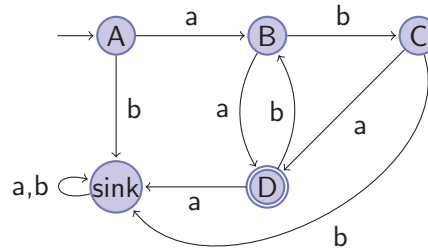


- Initial partition:
 $B_1 = \{D\}; B_2 = \{A, B, C, \text{sink}\}$
- Split B_2 by (a, B_1) because $\text{move}_d(B, a) \in B_1$ and $\text{move}_d(A, a) \notin B_1$
- First refinement:
 $B_1 = \{D\}; B_{21} = \{A, \text{sink}\}; B_{22} = \{B, C\}$

90 / 109

Training

EXAMPLE 1

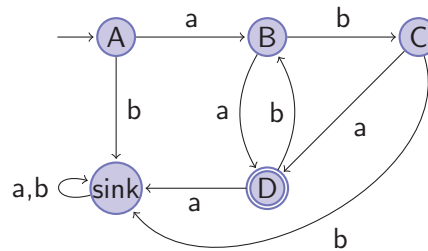


- Current partition:
 $B_1 = \{D\}$; $B_{21} = \{A, \text{sink}\}$; $B_{22} = \{B, C\}$
- Split B_{22} by (b, B_{21})
- Second refinement:
 $B_1 = \{D\}$; $B_{21} = \{A, \text{sink}\}$; $B_{221} = \{B\}$ $B_{222} = \{C\}$

91 / 109

Training

EXAMPLE 1

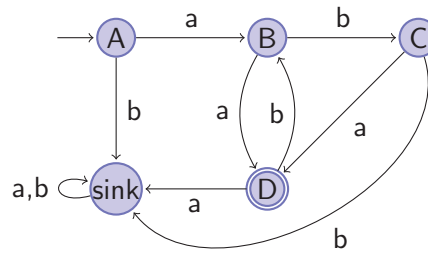


- Current partition:
 $B_1 = \{D\}$; $B_{21} = \{A, \text{sink}\}$; $B_{221} = \{B\}$ $B_{222} = \{C\}$
- Split B_{21} by (a, B_{221})
- Third and last refinement:
 $B_1 = \{D\}$; $B_{211} = \{A\}$; $B_{212} = \{\text{sink}\}$; $B_{221} = \{B\}$ $B_{222} = \{C\}$

92 / 109

Training

EXAMPLE 1



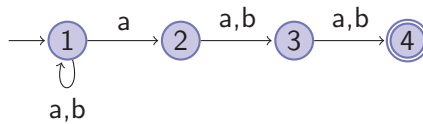
- Final partition:
 $B_1 = \{D\}$; $B_{211} = \{A\}$; $B_{212} = \{sink\}$; $B_{221} = \{B\}$ $B_{222} = \{C\}$
- Remove the sink and all the transitions to/from it, get the original DFA
- The original DFA was already minimal

93 / 109

Training

EXAMPLE 2

- Find the minimal equivalent DFA

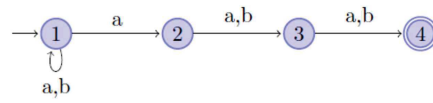


- First find the equivalent DFA
- Then minimize the DFA

94 / 109

Training

EXAMPLE 2



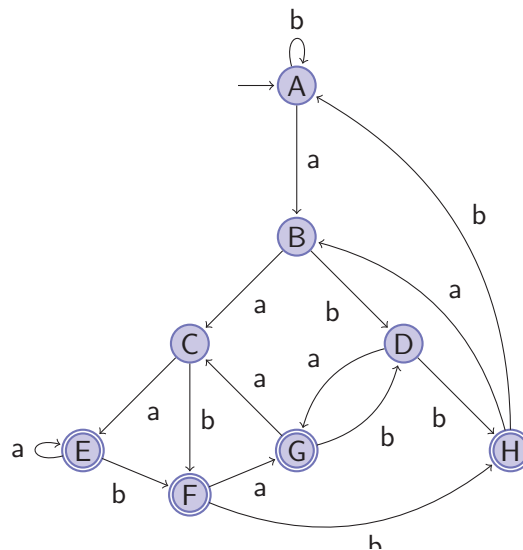
		a	b
A={1}	initial	B	A
B={1,2}		C	D
C={1,2,3}		E	F
D={1,3}		G	H
E={1,2,3,4}	final	E	F
F={1,3,4}	final	G	H
G={1,2,4}	final	C	D
H={1,4}	final	B	A

95 / 109

Training

EXAMPLE 2

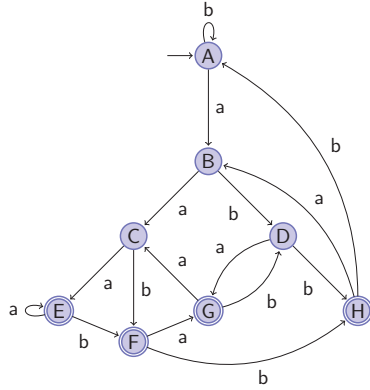
DFA resulting from the subset construction



96 / 109

Training

EXAMPLE 2

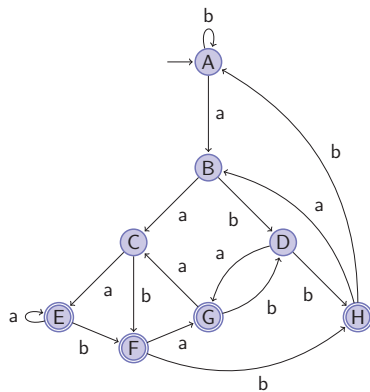


- Partition: $\{A, B, C, D\}; \{E, F, G, H\}$
- Observe: $\text{move}_d(A, a), \text{move}_d(C, a)$
- Partition: $\{A, B\}; \{C, D\}; \{E, F, G, H\}$
- Observe: $\text{move}_d(A, a), \text{move}_d(B, a)$
- Partition: $\{A\}; \{B\}; \{C, D\}; \{E, F, G, H\}$
- Observe: $\text{move}_d(G, b), \text{move}_d(H, b)$
- Partition: $\{A\}; \{B\}; \{C, D\}; \{E, F, H\}; \{G\}$

97 / 109

Training

EXAMPLE 2



- Partition: $\{A\}; \{B\}; \{C, D\}; \{E, F, H\}; \{G\}$
- Observe: $\text{move}_d(C, a), \text{move}_d(D, a)$
- Partition: $\{A\}; \{B\}; \{C\}; \{D\}; \{E, F, H\}; \{G\}$
- Observe: $\text{move}_d(E, a), \text{move}_d(H, a)$
- Partition: $\{A\}; \{B\}; \{C\}; \{D\}; \{E, F\}; \{H\}; \{G\}$
- Observe: $\text{move}_d(E, a), \text{move}_d(F, a)$
- Partition: $\{A\}; \{B\}; \{C\}; \{D\}; \{E\}; \{F\}; \{G\}; \{H\}$
- Initial DFA already minimal

98 / 109

Number of states of DFAs: worst case

LEMMA

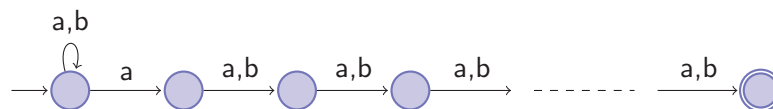
For each $n \in \mathbb{N}^+$ there is an NFA with $(n + 1)$ states whose minimal equivalent DFA has at least 2^n states and total transition function

99 / 109

Number of states of DFAs: worst case

PROOF

- Take $\mathcal{L} = \mathcal{L}((a \mid b)^* a (a \mid b)^{n-1})$
- There is an NFA accepting $\mathcal{L}$ which has exactly $n + 1$ states



- Suppose, by contradiction, that a minimal DFA $\mathcal{D}$ exists which accepts $\mathcal{L}$ and has $k < 2^n$ states
- There are exactly 2^n distinct words over $\{a, b\}$ whose length is n
- Then there are two paths in $\mathcal{D}$
 - Whose length is n
 - Spell, respectively, w_1 and w_2 with $w_1 \neq w_2$
 - Share at least one node

100 / 109

Number of states of DFAs: worst case

PROOF

- Call s the first node shared by the two paths spelling w_1 and w_2
- Then, for some x_1, x_2, x
 - Either $w_1 = x_1ax$ and $w_2 = x_2bx$, where x_1a (resp. x_2b) is the spelling of w_1 (resp. w_2) from the initial state to s
 - Or $w_1 = x_1bx$ and $w_2 = x_2ax$, where x_1b (resp. x_2a) is the spelling of w_1 (resp. w_2) from the initial state to s
- Wlog suppose that $w_1 = x_1ax$ and $w_2 = x_2bx$
- Then the state reached by the path x_1ab^{n-1} is final because $x_1ab^{n-1} \in \mathcal{L}(\mathcal{D})$
- Contradiction: that state cannot be final because it is also reached by x_2bb^{n-1} and $x_2bb^{n-1} \notin \mathcal{L}(\mathcal{D})$

101 / 109

Pumping Lemma for regular languages

LEMMA

Let $\mathcal{L}$ be a regular language. Then

- $\exists p \in \mathbb{N}^+$ such that
- $\forall z \in \mathcal{L}$ such that $|z| > p$
- $\exists u, v, w$ such that
 - $z = uvw$ and
 - $|uv| \leq p$ and
 - $|v| > 0$ and
 - $\forall i \in \mathbb{N}. uv^i w \in \mathcal{L}$

102 / 109

Pumping Lemma for regular languages

PROOF

- Let $\mathcal{L}$ be a regular language
- Then there exists a DFA $\mathcal{D} = (S, \mathcal{A}, \text{move}_d, s_0, F)$ such that $\mathcal{L} = \mathcal{L}(\mathcal{D})$
- Let $p = |S| - 1$
- Then all the paths from s_0 to some final state that traverse every state at most once have their lengths bounded by p
- Then, by $|z| > p$, for some $a_1, \dots, a_p$ and for some z' , $z = a_1 \cdots a_p z'$ and at least one state, say s^* , is traversed more than once along the path $a_1 \cdots a_p$

103 / 109

Pumping Lemma for regular languages

PROOF

- Then there is a cycle in $\mathcal{D}$ that goes from s^* to s^* and that spells $a_{i+1} \cdots a_j$ for some i and j such that $i < j \leq p$
- Then let
 - $u = a_1 \cdots a_i$
 - $v = a_{i+1} \cdots a_j$
 - $w = \begin{cases} z' & \text{if } j = p \\ a_{j+1} \cdots a_p z' & \text{if } j < p \end{cases}$
- Then
 - $|uv| \leq p$
 - The length of v is at least 1
 - $uv^i w$ is accepted by $\mathcal{D}$ for every $i \in \mathbb{N}$

104 / 109

Pumping Lemma for regular languages

WHAT FOR

- Show by contradiction that a language is not regular
 - Assume the language be regular
 - Show that not(thesis) is true
- Thesis:

$$\exists p \in \mathbb{N}^+. \forall z \in \mathcal{L}: |z| > p. \exists u, v, w. P$$
 where

$$P \equiv (z = uvw \text{ and } |uv| \leq p \text{ and } |v| > 0 \text{ and } \forall i \in \mathbb{N}. uv^i w \in \mathcal{L})$$
- not(thesis):

$$\forall p \in \mathbb{N}^+. \exists z \in \mathcal{L}: |z| > p. \forall u, v, w. Q$$
 where

$$Q \equiv (z = uvw \text{ and } |uv| \leq p \text{ and } |v| > 0) \text{ implies } (\exists i \in \mathbb{N}. uv^i w \notin \mathcal{L})$$

105 / 109

Pumping Lemma at work

LEMMA

$\mathcal{L} = \{a^n b^n \mid n > 0\}$ is not regular

106 / 109

Pumping Lemma at work

PROOF

- Suppose $\mathcal{L}$ is regular, and let p be an arbitrary positive integer
- Take $z = a^p b^p$
- Observe that $\forall u, v, w$ if ($z = uvw$ and $|uv| \leq p$ and $|v| > 0$)
 - Then v contains only a s, by $|uv| \leq p$
 - And v contains at least one a , by $|v| > 0$
- Then, for some $j > 0$, $uv^2w = a^p a^j b^p$
- Then $uv^2w \notin \mathcal{L}$ which contradicts the Pumping Lemma for regular languages

107 / 109

Training

- Let $\mathcal{L}_1$ be the language of words over $\{a, b\}$ with an odd number of occurrences of bs . Is $\mathcal{L}_1$ regular?
- Let $\mathcal{L}_2$ be the language of words over $\{a, b\}$ with an even number of occurrences of as . Is $\mathcal{L}_2$ regular?
- Let $\mathcal{L}_3$ be the language of words over $\{a, b\}$ with an even number of occurrences of as and an odd number of occurrences of bs . Is $\mathcal{L}_3$ regular?

108 / 109

Closure properties of regular languages

LEMMA

Regular languages are closed wrt

- union
- concatenation
- complementation
- intersection